Khoa Kỹ thuật Máy tính – Trường ĐH Công nghệ Thông tin – ĐHQG TP.HCM

CÔNG NGHỆ IoT VÀ ỨNG DỤNG (CE339) – HK1 2026–2027

THỰC HÀNH LAB 1 – CHUỖI ĐO TRÊN ESP32

Đọc cảm biến, đặc tính nhiễu, lọc số · Gắn với Chương 3 – Giao tiếp thế giới thực

Mục	Nội dung
Thời lượng	5 tiết (≈ 4 giờ), làm **cá nhân**
Chương liên quan	Chương 3 – Giao tiếp thế giới thực (Phần 2 Cảm biến, Phần 4 Lấy mẫu & ADC, Phần 5 Xử lý số trên MCU, Phần 6 Bus giao tiếp)
Nền tảng	Wokwi (ESP32 DevKit C v4, MPU6050, module NTC) · gcc trên máy tính hoặc Google Colab · Python 3 (numpy, matplotlib)
Trước buổi Lab	Nộp **Phiếu dự đoán TH1** (tính tay theo MSSV)
Sau buổi Lab	Nộp `TH1_<MSSV>_<HoTen>.zip` theo Mẫu báo cáo; vấn đáp 3–5 phút cuối buổi
Tệp khung (starter/)	`sketch.ino`, `filters.h`, `filters.c` (có TODO), `noise.h`, `noise.c`, `diagram.json`, `test_filters.c`, `du_lieu_mau.csv`, `phan_tich_th1.py` (có TODO)

1. Mục tiêu

Sau buổi thực hành, sinh viên có thể:

- **ĐỌC** gia tốc từ MPU6050 qua I2C (thanh ghi 0x3B) và điện áp NTC qua ADC; **KIỂM TRA** độ nhạy 8192 LSB/g ở dải ±4 g.
- **ĐO** thời gian một lần đọc I2C và **SO SÁNH** với ngân sách thời gian bus tính tay.
- **DỰ ĐOÁN** và **KIỂM CHỨNG** bằng số đo chiều thay đổi của mã ADC theo nhiệt độ (module NTC của Wokwi: NTC ở nửa dưới mạch chia áp → mã giảm khi T tăng); **VIẾT** hàm đổi mã ADC → °C bằng phương trình Beta.
- **TÍNH** trung bình, độ lệch chuẩn, histogram và ENOB từ 1000 mẫu.
- **TỰ CÀI ĐẶT** bằng C ba bộ lọc (trung bình trượt O(1), EMA, trung vị) và thống kê Welford; **KIỂM THỬ** bằng gcc trước khi nạp.
- **SO SÁNH** độ lệch chuẩn, độ trễ và khả năng chống xung của ba bộ lọc với dự đoán; **GIẢI THÍCH** mọi sai lệch > 20%.
- **HIỆU CHUẨN** 2 điểm một cảm biến bị lệch offset/độ lợi và **ĐÁNH GIÁ** sai số còn lại.

2. Liên hệ bài giảng và hệ thống xuyên suốt

Lab này hiện thực hoá **chuỗi đo** của nút giám sát máy bơm: MPU6050 ±4 g lấy mẫu 1 kHz (trên chip thật đặt DLPF 260 Hz – Wokwi không mô phỏng DLPF) cho rung động, NTC 10 kΩ (B = 3950) chia áp với điện trở 10 kΩ cho nhiệt độ vỏ động cơ, lọc **trung vị 5 → EMA**. Mỗi bước dưới đây ứng với một mục của Chương 3:

Việc trong Lab	Mục trong Chương 3 (đọc lại trước buổi Lab)
Đọc MPU6050, kiểm tra độ nhạy	Đọc datasheet: độ nhạy và nhiễu của MPU6050

Việc trong Lab	Mục trong Chương 3 (đọc lại trước buổi Lab)
Đo thời gian đọc I2C	I2C: hai dây, nhiều thiết bị, có địa chỉ · Ngân sách thời gian bus: 100 kHz có đủ cho 1 kHz?
Đổi mã ADC → °C	Ví dụ số: NTC và mạch chia áp
σ, histogram, ENOB	Lượng tử hoá · ENOB: ADC "12 bit" thực chất được bao nhiêu bit?
Ba bộ lọc bằng C	Lọc trung bình trượt · Lọc EMA · Lọc trung vị · So sánh ba bộ lọc trên cùng một tín hiệu
Hiệu chuẩn 2 điểm	Hiệu chuẩn 2 điểm – mô hình học máy đơn giản nhất
Thu 1000 mẫu vào RAM rồi in	Hệ quả: dữ liệu thô quá lớn để gửi đi · Tổng kết (UART 8N1: byte/s = baud/10)

Đối chiếu giáo trình: Hanes et al., *IoT Fundamentals*, Ch.3 (Smart Objects: sensors, MEMS); Lea, *IoT and Edge Computing for Architects*, Ch.3 (Sensors, Endpoints and Power Systems). Tài liệu kỹ thuật: datasheet MPU-6000/MPU-6050 (InvenSense), NXP UM10204 (I2C), ESP32 Technical Reference Manual (mục ADC).

3. Kiến thức nền cần dùng (tóm tắt)

Nội dung	Công thức	Ghi chú
NTC – phương trình Beta	$R(T) = R_0 \cdot \exp[B \cdot (1/T - 1/T_0)]$	R_0 = 10 kΩ, T_0 = 298,15 K, B = 3950 K. **T tính bằng Kelvin**
Chia áp, NTC ở nhánh trên	$V_{out} = V_{cc} \cdot R/(R_{NTC} + R)$	Như slide Ch.3; V_{out} TĂNG khi T tăng. **Không** phải cách nối của module Wokwi
Chia áp, NTC ở nhánh dưới	$V_{out} = V_{cc} \cdot R_{NTC}/(R_{NTC} + R)$	**Module Wokwi dùng cách này** (NTC nối GND): V_{out} GIẢM khi T tăng
ADC N bit	mã ≈ $4095 \cdot V_{out}/V_{cc}$; LSB = $V_{ref}/2^N$	Wokwi: ADC **lý tưởng** 0–3,3 V ↦ 0–4095, không nhiễu, không phi tuyến (ESP32 thật thì có cả hai)
Nhiễu lượng tử, ENOB	σ_q = LSB/√12 ≈ 0,29 LSB; ENOB ≈ N − $\log_2(\sigma_{đo} \cdot \sqrt{12})$	$\sigma_{đo}$ tính bằng LSB
Nhiễu tổng (độc lập)	$\sigma_{tổng} = \sqrt{(\sigma_1^2 + \sigma_2^2)}$	Nhiễu mô phỏng làm tròn về số nguyên thêm 1/12 LSB^2
Trung bình trượt N	$\sigma_{MA} = \sigma/\sqrt{N}$; d_{50} = (N − 1)/2 mẫu (N lẻ)	Xung biên độ A → N mẫu lệch A/N
EMA hệ số α	$\sigma_{EMA} = \sigma \cdot \sqrt{(\alpha/(2 - \alpha))}$; $N_{tđ} \approx 2/\alpha - 1$	$d_{50} = \lceil \ln 0{,}5 / \ln(1 - \alpha) \rceil - 1$ (≈ 0,69/α); xung A → $\alpha A(1 - \alpha)^j$
Trung vị N (lẻ)	$\sigma_{med} \approx 1{,}25 \cdot \sigma/\sqrt{N}$ (nhiễu Gauss, gần đúng)	Loại xung ngắn hơn ⌈N/2⌉ mẫu; d_{50} = (N − 1)/2
Giao dịch I2C đọc n byte	$t \approx [(n + 3) \cdot 9 + 3] / f_{SCL}$	3 byte địa chỉ/thanh ghi; 9 xung/byte; +3 cho START/RESTART/STOP
UART 8N1	byte/s = baud/10	115200 baud → 11 520 byte/s
Hiệu chuẩn 2 điểm	$g = (T_H - T_L)/(x_H - x_L)$; $T = g \cdot (x - x_L) + T_L$	Sửa offset và độ lợi; không sửa nhiễu, không sửa phi tuyến

Định nghĩa độ trễ d_{50} dùng trong Lab: cho đầu vào bậc thang KHÔNG nhiễu (nền $x_0 \rightarrow x_0 + S$ tại mẫu n_0), d_{50} là số mẫu từ n_0 đến mẫu **đầu tiên** có đầu ra $\geq x_0 + S/2$. Ví dụ MA N = 5: đầu ra tại n_0, n_0+1, n_0+2 là S/5, 2S/5, 3S/5 → $d_{50} = 2$.

Vì sao phải cộng nhiễu bằng code? Wokwi cho giá trị cảm biến đúng bằng vị trí thanh trượt: ADC mô phỏng lý tưởng (không nhiễu, không phi tuyến), MPU6050 không nhiễu và không rung. Mọi nhiễu trong bài này là **nhiễu mô phỏng cộng bằng code**, phải ghi rõ điều đó trong báo cáo. Tệp `noise.c` (cho sẵn) sinh nhiễu Gauss và xung nhiễu tất định với hạt giống = MSSV, nên cùng MSSV luôn ra cùng một chuỗi nhiễu – giảng viên kiểm tra lại được số liệu của bạn.

Tham số cá nhân của bạn

Quy ước: **d** = chữ số cuối MSSV, **c** = chữ số áp cuối. Bạn chỉ cần sửa dòng `#define MSSV` trong sketch – mọi tham số tự tính và được in ra ở đầu log. Đối chiếu với Phiếu dự đoán.

Tham số	Công thức theo MSSV	Ví dụ 23520147 (d = 7, c = 4)
Nhiệt độ đặt T_SET	20 + 3c (°C)	32 °C
Nhiễu Gauss mô phỏng σ_n	4 + d (LSB)	11 LSB
Cửa sổ trung bình trượt N	5 + 2·(d mod 3)	7
Hệ số EMA α	0,1·(1 + c mod 3)	0,2
Cửa sổ trung vị	5 (như hệ thống xuyên suốt)	5
Chu kỳ xung nhiễu K, biên độ A	K = 50 + 10d mẫu; A = 1000 LSB	K = 120
Bậc thang (CHE_DO 3)	S = 400 LSB tại n_0 = 500, không nhiễu	S = 400, n_0 = 500
Xung nhịp I2C f_SCL	100·(1 + c mod 4) kHz	100 kHz
Cảm biến lệch (CHE_DO 5)	độ lợi 1,02 + 0,01d; offset 0,5 + 0,3c (°C)	1,09; 1,7 °C

4. Chuẩn bị

- Tài khoản **wokwi.com** (đăng nhập để lưu project và lấy link chia sẻ). Tạo project mới loại **ESP32** (Arduino).
- **gcc**: Linux/WSL/macOS có sẵn; Windows dùng MSYS2/MinGW; hoặc Google Colab (ô lệnh `!gcc ...`).
- **Python 3** với numpy, matplotlib (máy cá nhân hoặc Colab).
- Tải thư mục `starter/`; đọc lại các mục Chương 3 ở bảng mục 2; **nộp Phiếu dự đoán trước buổi Lab**.

5. Sơ đồ nối dây trên Wokwi

Chép nội dung `diagram.json` của thư mục starter vào tab **diagram.json** của project (thay toàn bộ). Thêm các tệp `filters.h`, `filters.c`, `noise.h`, `noise.c` bằng nút tạo tệp mới (+) trong trình soạn thảo Wokwi.

5.1 Linh kiện Wokwi dùng trong bài

Linh kiện	Part id (diagram.json)	Chân nối (linh kiện → ESP32)	Thuộc tính cần đặt
ESP32 DevKit C v4	`board-esp32-devkit-c-v4` (id `esp`)	Dùng `esp:3V3`, `esp:GND.1`, `esp:GND.2`, `esp:21`, `esp:22`, `esp:34` (không có chân tên "GND" trần)	—
Cảm biến gia tốc MPU6050	`wokwi-mpu6050`	VCC → 3V3 · GND → GND.2 · SDA → 21 · SCL → 22 (AD0 để trống → địa chỉ 0x68)	`accelX/Y/Z` (g) mặc định 0, 0, 1 – chỉnh bằng thanh trượt khi chạy
Module NTC	`wokwi-ntc-temperature-sensor`	VCC → 3V3 · GND → GND.1 · OUT → 34 (ADC1_CH6; ADC2 xung đột Wi-Fi)	`temperature` (°C, mặc định 24), `beta` = 3950 (giữ nguyên)

Cách nối bên trong module NTC của Wokwi (theo tài liệu chính thức của Wokwi): 3V3 — điện trở cố định 10 kΩ — **OUT** — NTC 10 kΩ — GND. NTC nằm ở **nửa dưới** mạch chia áp nên V_out = V_cc·R_NTC/(R_NTC + R): nhiệt độ tăng → R_NTC giảm → **mã ADC GIẢM**. Ở Bước 2 bạn **kiểm chứng** điều này bằng số đo và so với mã đã dự đoán trong Phiếu.

5.2 Giới hạn của Wokwi ảnh hưởng tới bài này

- **ADC lý tưởng:** không nhiễu, không phi tuyến, không lệch offset. σ đo ở CHE_DO 0 bằng 0 → ENOB "12 bit" chỉ là lý thuyết. Nhiễu ở CHE_DO 1–2 và bậc thang ở CHE_DO 3 đều do code cộng vào (`noise.c`).
- **MPU6050 không nhiễu, không rung:** giá trị đúng bằng thanh trượt. Wokwi **không** mô phỏng bộ lọc số DLPF, FIFO, tốc độ lấy mẫu SMPLRT_DIV và không dùng được ngắt data-ready (chân INT). Mã khung vẫn ghi thanh ghi 0x1A/0x19 cho giống chip thật, nhưng trên Wokwi chúng **không có tác dụng**; nhịp 100 Hz/1 kHz do ESP32 tự định thời bằng `micros()`.
- **Thời gian mô phỏng:** `micros()`/`millis()` là thời gian mô phỏng. Nếu thanh tốc độ mô phỏng của Wokwi dưới 100%, một lần chạy "10 s" sẽ mất lâu hơn 10 s thực; số đo thời gian vẫn tính theo thời gian mô phỏng.
- **Thời gian bus I2C và UART** trên Wokwi chưa chắc khớp chip thật từng µs – đó là lý do bạn so sánh số đo với dự đoán và giải thích chênh lệch, không coi số Wokwi là "chân lý".
- **Lấy dữ liệu ra:** chỉ qua Serial Monitor (chọn hết, copy, dán vào tệp .txt). Mỗi lần chạy in khoảng 1000 dòng – đủ nhỏ để chép; chờ dòng `# XONG` rồi mới chép.

6. Các bước thực hiện

Thời gian gợi ý ghi trong ngoặc. Sau mỗi lần chạy, chép **toàn bộ** nội dung Serial Monitor vào tệp văn bản (ví dụ `che_do1.txt`) – đó là dữ liệu thô bạn phải nộp.

Bước 0 – Tạo project, chạy thử (10 phút)

1. Tạo project ESP32, chép `sketch.ino`, `diagram.json`, thêm 4 tệp C/H. Sửa `#define MSSV` thành MSSV của bạn.
2. Để `CHE_DO 0`, bấm chạy. Kiểm tra dòng `# MSSV=… d=… c=…` và bảng tham số in ra khớp với Phiếu dự đoán.

```
#define MSSV 23520147UL        // <-- sửa thành MSSV của bạn
#define CHE_DO 1   // 0: ADC thô | 1: + nhiễu Gauss | 2: + nhiễu Gauss + xung
                   // 3: bậc thang không nhiễu | 4: MPU6050 1 kHz vào RAM rồi in
                   // 5: hiệu chuẩn 2 điểm
const uint32_t D = MSSV % 10;          // chữ số cuối
const uint32_t C = (MSSV / 10) % 10;   // chữ số áp cuối
const float    SIGMA_N = 4.0f + D;     // nhiễu Gauss mô phỏng (LSB)
const uint8_t  N_MA    = 5 + 2 * (D % 3);
const float    ALPHA   = 0.1f * (1 + C % 3);
```

Bước 1 – MPU6050 qua I2C (25 phút)

1. Đọc dòng `# I2C scan:` – phải thấy `0x68`. Không thấy: kiểm tra dây SDA/SCL.
2. **TODO T2:** điền `ACC_LSB_PER_G` bằng độ nhạy ở dải ±4 g (bảng trong mục "Đọc datasheet…"). Hàm `mpu_init()` đã ghi `ACCEL_CONFIG = 0x08` (AFS_SEL = 1).
3. Bấm vào MPU6050 trên sơ đồ, đặt thanh trượt gia tốc x = 0, y = 0, z = 1 g. Chạy lại: `az` phải ≈ 8192 LSB và |a| ≈ 1,000 g. Thử z = 2 g, −1 g; ghi vào báo cáo.
4. Ghi `# I2C doc 6 byte: … us/lan` (trung bình 100 lần đọc). So sánh với t_I2C dự đoán ở Phiếu (84 bit / f_SCL). Đổi tạm `I2C_HZ` sang 400 kHz và 100 kHz, ghi lại 3 giá trị.

Câu hỏi giữa buổi 1: thời gian đo được lớn hơn hay nhỏ hơn 84/f_SCL? Phần chênh lệch đến từ đâu (thư viện Wire, ngắt, cách Wokwi mô phỏng bus)? Với f_SCL của bạn, đọc 1000 mẫu/s chiếm bao nhiêu % thời gian CPU?

Bước 2 – NTC: xác định cách nối và đổi sang °C (25 phút)

1. Bấm vào module NTC, đặt nhiệt độ = T_SET của bạn. Chạy CHE_DO 0 và đọc cột x (hoặc tạm in `analogRead(PIN_NTC)` trong `setup()`). Ghi mã ADC và so với mã dự đoán ở Phiếu (câu 2b).
2. Đặt lần lượt 0 °C và 60 °C, ghi mã ADC. **Kiểm chứng**: mã phải **giảm** khi nhiệt độ tăng (NTC ở nửa dưới) và khớp mã dự đoán ở Phiếu (câu 2b) trong vài LSB. Lệch nhiều hơn → xem lại dây OUT (phải vào GPIO34) và công thức của bạn.
3. **TODO T1:** viết `ntc_code_to_celsius()` theo gợi ý trong mã. Kiểm tra: dòng `# ENOB=… T(mean_x)=…` ở CHE_DO 0 phải cho T sai khác nhiệt độ đặt < 0,1 °C tại 0 °C, T_SET và 60 °C.

```
float ntc_code_to_celsius(float code) {
  // TODO T1: đổi mã ADC (0..4095) -> °C.
  //   (a) x = code / 4095 là tỉ số V_out/V_cc; giới hạn x trong (0,001; 0,999)
  //   (b) Tìm R_ntc từ mạch chia áp: NTC ở nửa DƯỚI (nối GND)
  //   (c) Phương trình Beta: 1/T = 1/T0 + ln(R_ntc/R0)/B   (Kelvin!)
  return 0.0f;
```

```
}
```

Bước 3 – Đặc tính nhiễu: 1000 mẫu (20 phút)

1. Chạy `CHE_DO 0` (không cộng nhiễu). Độ lệch chuẩn cột x là bao nhiêu? ENOB tính theo công thức có ý nghĩa không? Giải thích bằng một câu về bản chất ADC của Wokwi so với ADC thật của ESP32.
2. Chạy `CHE_DO 1` (nhiễu Gauss σ_n **cộng bằng code**): 1000 mẫu ở 100 Hz (10 s thời gian mô phỏng). Lưu log thành `che_do1.txt`.
3. Trong `phan_tich_th1.py`, viết **TODO** `thong_ke()` và `enob()`. Chạy thử với tệp mẫu để kiểm tra (xem Bước 5). Histogram phải có dạng chuông quanh mã ADC của T_SET.

Bước 4 – Tự cài đặt ba bộ lọc bằng C, kiểm thử trên PC (50 phút)

Đây là **phần cốt lõi** của Lab. Viết 6 hàm có TODO trong `filters.c`. Mỗi hàm đều có chú thích nêu yêu cầu. Không đổi giao diện trong `filters.h`.

```
float ma_update(ma_t *f, int32_t x) {
    /* TODO(1): cài đặt O(1) bằng TỔNG CHẠY – KHÔNG cộng lại N phần tử mỗi lần.
       - cập nhật f->sum (cộng mẫu mới, trừ mẫu cũ nhất đang ở f->buf[f->idx]);
       - ghi x vào bộ đệm vòng, tăng f->idx theo modulo f->N;
       - f->count tăng tới N rồi dừng;
       - trả về trung bình của f->count mẫu hiện có (lúc khởi động count < N). */
    return 0.0f;
}
float   ema_update(ema_t *f, float x);     /* TODO(2): mẫu đầu tiên gán y = x   */
int32_t med_update(med_t *f, int32_t x);   /* TODO(3): sắp xếp BẢN SAO cửa sổ  */
void    stats_add(stats_t *s, double x);   /* TODO(4): Welford, không lưu mảng  */
double  stats_std(const stats_t *s);       /* TODO(5): chia n - 1               */
double  enob_from_sigma(double s, int n);  /* TODO(6)                           */
```

Biên dịch và chạy bộ kiểm thử (đã viết sẵn) với **N và α của bạn**:

```
gcc -O2 -Wall -o test_filters test_filters.c filters.c noise.c -lm
./test_filters du_lieu_mau.csv <N_MA> <ALPHA>        # ví dụ: ./test_filters
du_lieu_mau.csv 7 0.2
```

Bộ kiểm thử gồm: ví dụ tính tay trong slide "Lọc trung vị" (nhân 10 thành số nguyên), Welford, σ trước/sau lọc trên dữ liệu mẫu (σ ≈ 8 LSB), đếm xung, và độ trễ d_{50} trên bậc thang. Chỉ khi hiện `Tat ca kiem tra OK` mới chép `filters.c` sang Wokwi. Chụp màn hình kết quả cho báo cáo.

> **Câu hỏi giữa buổi 2:** (a) Vì sao `sum` là `int32_t` chứ không phải `float`? (b) Vì sao không sắp xếp trực tiếp `f->buf` của bộ lọc trung vị? (c) Với N = 9 và mã ADC tối đa 4095, `sum` lớn nhất bao nhiêu – có tràn int16 không?

Bước 5 – Đo và so sánh ba bộ lọc (40 phút)

1. Chạy lần lượt `CHE_DO 1`, `2`, `3`; lưu `che_do1.txt`, `che_do2.txt`, `che_do3.txt`. Mỗi lần chạy mất 10 s thời gian mô phỏng (lâu hơn nếu tốc độ mô phỏng < 100%).

2. Điền dự đoán vào `DU_DOAN` trong `phan_tich_th1.py`, viết nốt TODO `do_tre_d50()` và `dem_lech()`, rồi chạy:

```
python3 phan_tich_th1.py che_do1.txt che_do2.txt che_do3.txt
```

3. Chương trình in bảng Đo – Dự đoán – Lệch và đánh dấu mọi dòng lệch > 20%; lưu `ket_qua_th1.csv`, `hinh_th1.png` (histogram, xung nhiễu, bậc thang). Đối chiếu với dòng `# sigma: …` do chính ESP32 tính bằng Welford – hai cách tính phải khớp.

4. Giải thích từng sai lệch > 20%. Gợi ý các nguồn sai lệch hợp lệ: công thức trung vị chỉ gần đúng với N nhỏ; 950 mẫu cho ước lượng σ có sai số thống kê cỡ $1/\sqrt{(2 \cdot 950)} \approx 2{,}3\%$; nhiễu cộng thêm vào xung làm vài mẫu vượt/không vượt ngưỡng.

Bước 6 – MPU6050 1 kHz vào RAM rồi mới in (20 phút)

`CHE_DO 4` đọc 1000 mẫu gia tốc ở 1 kHz (nhịp do vòng chờ `micros()` của ESP32 tạo ra) vào mảng `acc_buf[1000][3]`, sau đó mới in ra Serial. Ghi `capture_us` và `print_us`, so với dự đoán (câu 5 của Phiếu). Trả lời: vì sao **không thể** in từng mẫu ngay khi đọc ở 1 kHz với 115200 baud? Mảng này chiếm bao nhiêu byte và bao nhiêu % SRAM 520 KB của ESP32?

> Trên Wokwi, MPU6050 không nhiễu → 1000 dòng gần như giống hệt nhau (ví dụ `n,0,0,8192`). Đó là bình thường: bước này đo **thời gian và bộ nhớ**, không phân tích rung. Nếu f_SCL của bạn là 100 kHz, một lần đọc (≥ 840 µs + chi phí thư viện) có thể không vừa chu kỳ 1 ms → `capture_us` > 1 000 000: hãy giải thích bằng ngân sách thời gian bus.

Bước 7 – Hiệu chuẩn 2 điểm (25 phút)

1. `CHE_DO 5` mô phỏng một cảm biến bị lệch: giá trị đọc `x_doc = độ lợi·T + offset` (tham số theo MSSV). Mỗi 2 s in trung bình 200 mẫu.

2. Đặt NTC ở **0 °C**, ghi x_L; đặt **60 °C**, ghi x_H. Tính g và x_0 = x_L (điểm dưới là 0 °C nên T_L = 0).

3. **TODO T3:** điền `CAL_G`, `CAL_X0` và dòng áp dụng hiệu chuẩn. Kiểm tra tại T_SET và tại 20 °C: sai số sau hiệu chuẩn bao nhiêu? So với trước hiệu chuẩn?

Bước 8 – Vấn đáp (3–5 phút/SV)

GV chọn một hàm trong `filters.c` của bạn để bạn giải thích từng dòng, rồi đổi một tham số (N, α, K, f_SCL hoặc cách nối NTC) và yêu cầu dự đoán nhanh kết quả mới trước khi chạy.

7. Câu hỏi kiểm tra giữa buổi (trả lời trong báo cáo)

1. Thời gian đọc I2C đo được so với 84/f_SCL: lệch bao nhiêu %, vì sao? (Bước 1)
2. Số đo nào của bạn xác nhận NTC trong module Wokwi nằm ở nửa dưới mạch chia áp? Nếu một module khác nối NTC ở nửa trên (như slide), mã ADC tại 0 °C và 60 °C sẽ là bao nhiêu? (Bước 2)
3. Vì sao σ ở CHE_DO 0 bằng 0? Một ADC thật của ESP32 sẽ cho kết quả thế nào? (Bước 3)

4. Ba câu hỏi ở hộp "Câu hỏi giữa buổi 2". (Bước 4)
5. EMA của bạn có σ gần bằng MA với $N_td = 2/\alpha - 1$, nhưng d_{50} có bằng nhau không? Vì sao? (Bước 5)
6. Vì sao trung vị gần như miễn nhiễm với xung còn MA thì không? Cái giá phải trả là gì? (Bước 5)

8. Sản phẩm phải nộp

- Báo cáo PDF theo **Mẫu báo cáo TH1** (đủ bảng số liệu, 3 hình, câu trả lời, nhật ký AI).
- Mã nguồn: `sketch.ino`, `filters.c` (đã hoàn thiện), `phan_tich_th1.py`.
- Dữ liệu thô: `che_do0.txt` … `che_do5.txt` (log Serial), `ket_qua_th1.csv`, `hinh_th1.png`, ảnh chụp kết quả `test_filters`.
- Link Wokwi của project (chế độ chia sẻ công khai hoặc "unlisted").
- Đóng gói thành `TH1_<MSSV>_<HoTen>.zip` (họ tên không dấu, không khoảng trắng).

9. Bài nâng cao (điểm cộng)

Nâng cao 1 – EMA số nguyên không phép chia

Cài `acc += x - (acc >> K); y = acc >> K;` ($\alpha = 1/2^K$, slide "Lọc EMA"). Chọn K sao cho $1/2^K$ gần α của bạn nhất. So sánh σ và d_{50} với bản số thực; đo thời gian chạy 1000 lần mỗi bản bằng `micros()` trên ESP32. Giải thích sai số do cắt bit (vì sao `acc` phải giữ K bit phần lẻ?).

Nâng cao 2 – Hiệu chuẩn 2 điểm không sửa được phi tuyến

Giả sử bạn nhầm loại NTC và dùng B = 3435 K trong `ntc_code_to_celsius()` (cảm biến thật vẫn là 3950 K). Dự đoán bằng Python nhiệt độ đọc được ở 0, 20, 40, 60 °C; hiệu chuẩn 2 điểm tại 0 và 60 °C; tính sai số còn lại lớn nhất trong dải và vị trí của nó. Kiểm chứng trên Wokwi. Kết luận: khi nào cần hiệu chuẩn nhiều điểm (hồi quy – Chương 10)?

Nâng cao 3 – Phát hiện xung bằng z-score trượt O(1)

Cài z-score trượt dùng tổng chạy Σx và Σx^2 (slide "Ví dụ: phát hiện điểm bất thường bằng z-score trượt"), cửa sổ W = 32, ngưỡng $|z| > 4$ trên dữ liệu CHE_DO 2. Đếm số xung bắt được và số báo động giả; thử 3 ngưỡng và vẽ đánh đổi. Lưu ý: tính σ từ Σx^2 bằng float có thể mất chính xác – giải thích và đề xuất cách khắc phục.

10. Lỗi thường gặp

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
Không thấy 0x68 khi quét I2C	Nhầm SDA/SCL; quên `Wire.begin(21, 22)`	Kiểm tra diagram.json và dòng Wire.begin
az ≈ 16384 thay vì 8192	Chưa ghi ACCEL_CONFIG hoặc ghi sai giá trị	AFS_SEL nằm ở bit 4:3 → ±4 g là 0x08
Giá trị gia tốc nhảy lung tung	Viết `(Wire.read() << 8) | Wire.read()` – thứ tự tính hai lời gọi không xác định trong C/C++	Đọc từng byte vào biến trước (như hàm cho sẵn)

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
T sai hàng chục độ hoặc ngược chiều thanh trượt	Dùng °C thay cho Kelvin trong phương trình Beta; dùng công thức NTC nhánh trên (slide) thay cho nhánh dưới của module Wokwi	T_0 = 298,15 K; R_NTC = R·x/(1 − x)
MA trôi dần sau nhiều giờ	`sum` kiểu float tích luỹ sai số làm tròn	Dùng int32_t cho tổng chạy
Trung vị cho kết quả sai từ mẫu thứ N+1	Sắp xếp trực tiếp bộ đệm vòng làm hỏng thứ tự thời gian	Sắp xếp bản sao
σ của EMA nhỏ bất thường ngay đầu	Tính cả giai đoạn khởi động bộ lọc	Bỏ 50 mẫu đầu (BO_QUA = 50)
Log thiếu dòng, CSV lỗi	Chép Serial Monitor khi chưa chạy xong	Chờ dòng `# XONG` rồi mới chép
test_filters không biên dịch	Thiếu `-lm`, hoặc sửa filters.h	Giữ nguyên filters.h; thêm -lm

Quy định sử dụng AI và cách tính điểm

Phần việc	Vùng	Được phép
Phiếu dự đoán	VÀNG	Hỏi AI để hiểu khái niệm; tự tính, tự trình bày. Ghi nhật ký AI nếu có hỏi
Mã nguồn (filters.c, sketch, Python)	XANH	Được dùng AI sinh mã, nhưng phải khai báo phần nào do AI sinh và giải thích được từng dòng khi vấn đáp
Vấn đáp cuối buổi	ĐỎ	Không dùng thiết bị; giải thích mã của chính mình, dự đoán kết quả khi GV đổi 1 tham số

Điểm Lab = điểm báo cáo × hệ số vấn đáp (1,0 / 0,8 / 0,5 / 0,2). Điểm báo cáo theo rubric A3: Phiếu dự đoán **20%** · Thực hiện và số liệu **50%** · Giải thích sai lệch **30%**. Mọi sai lệch giữa đo và dự đoán lớn hơn 20% đều phải được giải thích; một sai lệch được giải thích đúng có giá trị hơn một kết quả "khớp" mà không hiểu vì sao.